PROGRAMACIÓN II · UNIDAD 1 · CLASE 9

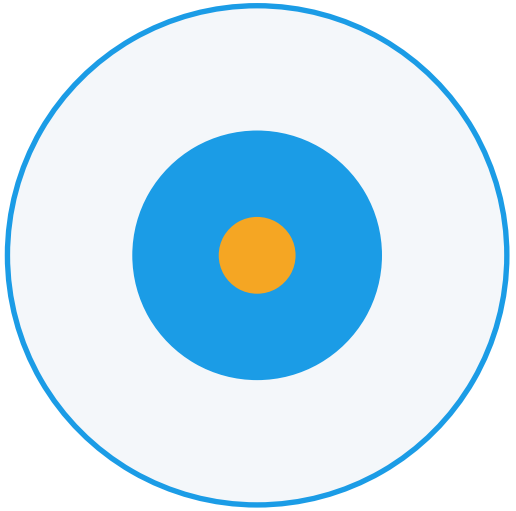
Diccionarios, conjuntos y miembros de clase

Bloque 5 · Programación Orientada a Objetos en C#

> *Completar las colecciones principales y presentar los miembros estáticos y las constantes.*

Objetivo de la clase

Completar las **colecciones principales** y presentar los **miembros estáticos** y las **constantes**.



Qué veremos hoy

1. Dictionary<K,V>
2. HashSet<T>
3. Miembros static
4. Constantes (static readonly)

1 Dictionary<K,V>

Colección de pares clave → valor; cada valor se accede por su clave (única).

Dictionary.cs

```
using System.Collections.Generic;

Dictionary<string, string> agenda =
    new Dictionary<string, string>();

agenda.Add("Ada", "1234");          // alta
agenda["Alan"] = "5678";            // acceso por clave
agenda.Remove("Alan");              // baja por clave

if (agenda.ContainsKey("Ada"))      // ¿existe la clave?
    Console.WriteLine(agenda["Ada"]);

foreach (var par in agenda)         // recorrido
    Console.WriteLine($"{par.Key} = {par.Value}");
```

- Pares clave → valor; la clave es única.
- Add y acceso por clave con [ ].
- ContainsKey: verifica si la clave existe.
- Remove: elimina por clave.
- foreach: recorre pares (Key / Value).

Apunte Unidad 1, pág. 37-38

2 HashSet<T>

Colección de elementos únicos: no admite duplicados.

HashSet.cs

```
using System.Collections.Generic;

HashSet<int> numbers = new HashSet<int>();

numbers.Add(1);      // true  (se agregó)
numbers.Add(2);      // true
numbers.Add(1);      // false (ya estaba)

Console.WriteLine(numbers.Count);   // 2
```

- Cada elemento es único (sin duplicados).
- Add devuelve true si se agregó, false si ya estaba.
- Ideal para trabajar con conjuntos.

Con objetos propios, **la unicidad depende de Equals() y GetHashCode()** → lo vemos en la próxima diapositiva.

Apunte Unidad 1, pág. 38-40

2 Unicidad en objetos propios

Para tipos propios hay que definir cuándo dos objetos son “iguales”.

Persona.cs

```
public class Persona
{
    public int Id { get; set; }

    public override bool Equals(object obj)
    {
        return obj is Persona p && p.Id == Id;
    }

    public override int GetHashCode()
    {
        return Id.GetHashCode();
    }
}
```

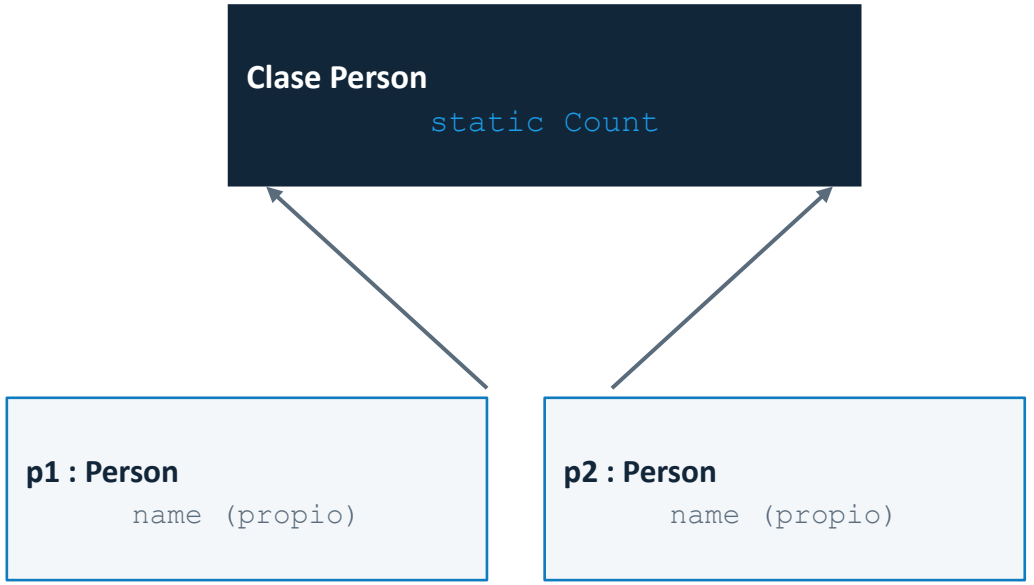
- Por defecto se compara por referencia: dos objetos distintos nunca son iguales.
- Sobreescribiendo Equals() y GetHashCode() se compara por contenido (aquí, por Id).

El apunte implementa **`GenericSet<T>`**, un conjunto genérico propio que usa estos métodos para rechazar duplicados (ej.: GenericSet<Persona> con clave por Id).

Apunte Unidad 1, pág. 38-41

3 static: miembros de clase vs. de instancia

- Un miembro de instancia existe en cada objeto: cada uno tiene el suyo.
- Un miembro static pertenece a la clase: hay una sola copia, compartida por todas las instancias.
- Se accede por el nombre de la clase (Clase.Miembro), no a través de una instancia.



Count se comparte entre p1 y p2; name es propio de cada objeto.

Apunte Unidad 1, pág. 42-44

3 static: la regla de acceso

Person.cs

```
public class Person
{
    public static int Count = 0;    // de clase
    private string name;            // de instancia

    public Person(string name)
    {
        this.name = name;
        Count++;                    // suma a la cuenta
    }
}

Console.WriteLine(Person.Count);    // por la clase
```

✓ **no estático → estático**

Un método de instancia puede usar un miembro static.

✗ **estático → no estático**

Un método static no puede usar miembros de instancia: no hay this.

Apunte Unidad 1, pág. 42-44

4 Constantes: readonly + static

Una constante se arma combinando **`readonly`** y **`static`**.

Config.cs

```
public class Config
{
    public static readonly int MaxUsuarios = 100;
    public static readonly string Version = "1.0";
}

Console.WriteLine(Config.MaxUsuarios);
```

- readonly: se asigna una sola vez (en la declaración o en el constructor) y no cambia.
- static: pertenece a la clase; una sola copia compartida.
- static readonly: constante a nivel de clase, compartida y de solo lectura.

Apunte Unidad 1, pág. 44-45

EN SÍNTESIS

- Dictionary<K,V>: pares clave → valor; Add, acceso por clave, ContainsKey, Remove y foreach.
- HashSet<T>: elementos únicos; Add devuelve true/false.
- En objetos propios, la unicidad exige sobreescribir Equals() y GetHashCode() (GenericSet<T>).
- static: miembro de clase (una copia compartida); no estático → estático sí, al revés no.
- Constantes: static readonly, a nivel de clase, compartidas y de solo lectura.

Próxima clase (práctica): Dictionary y HashSet, GenericSet<Persona> con clave por Id y una constante readonly static.